

Escalonamento

Sistemas em Lote e Interativos

Prof. Dr. Márcio Castro
marcio.castro@ufsc.br



1

Conceitos básicos

Quando o escalonamento é necessário?

1. Ao criar um processo

- Executar o processo pai ou filho?

2. Ao término do processo

- Qual processo “pronto” deve ser escolhido?

3. Ao passar para o estado bloqueado

- Qual processo “pronto” deve ser escolhido?

4. Na ocorrência de uma interrupção

- Continuar a execução do processo atual?
- Executar o processo que acabou de ficar pronto?
- Escolher outro processo?

Tipos de algoritmos de escalonamento

- **Algoritmos não preemptivos**

- O processo executa **até que seja bloqueado ou até que termine**
- Processos podem executar por horas sem serem suspensos

- **Algoritmos preemptivos**

- O processo executa por um **tempo máximo fixado**
- Ao final do intervalo de tempo o processo é **suspenso** e o escalonador escolhe outro processo para executar
- Requer **interrupção de relógio**

Escalonamento e os tipos de sistema

■ Sistemas em lote

- Usuários submetem *jobs* a serem executados
- Usuários não aguardam por uma resposta rápida
- Escalonadores **não preemptivos são em geral aceitáveis**

■ Sistemas interativos

- Usuários interagem diretamente com o sistema, executando comandos e aguardando resultados
- Escalonadores **preemptivos são essenciais**
- Evitam que processos se apossem da CPU

Objetivos dos algoritmos de escalonamento

- **Justiça**

- Dar a cada processo uma porção justa da CPU

- **Eficiência**

- Manter a CPU ocupada 100% do tempo quando houver demanda

- **Vazão (*throughput*)**

- Maximizar o número de *jobs* por unidade de tempo

- **Tempo de retorno (*turnaround time*)**

- Minimizar o tempo entre a submissão e o término de um *job* (importante para sistemas em lote)

- **Tempo de resposta**

- Minimizar o tempo entre a emissão de um comando e a obtenção do resultado (importante para sistemas interativos)

Objetivos dos algoritmos de escalonamento

- **Nem sempre é possível atingir todos os objetivos**
 - Um algoritmo de escalonamento que maximize a vazão pode não necessariamente minimizar o tempo de retorno
- **Exemplo**
 - Considere uma mistura de *jobs* **curtos** e **longos**
 - Um escalonador que execute sempre *jobs* curtos primeiro pode conseguir uma **excelente vazão** mas à custa de um **enorme tempo de retorno para os *jobs* longos**

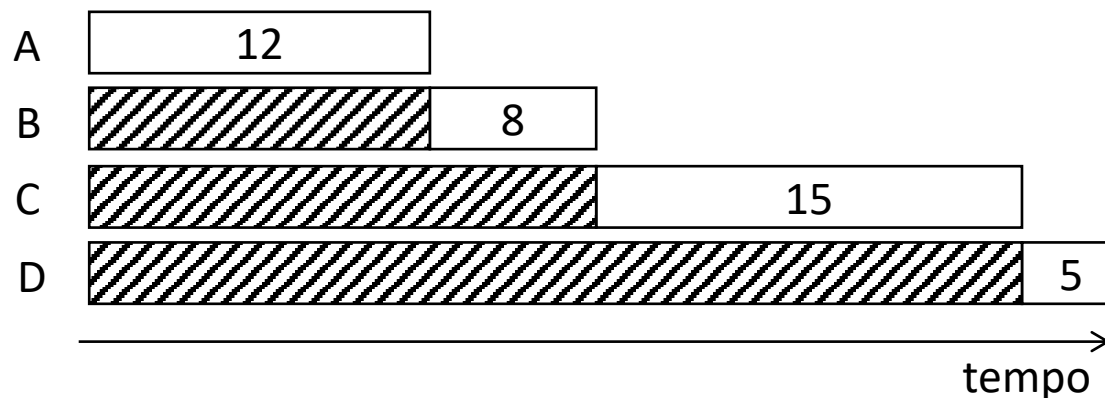
2

Escalonamento em sistemas em lote

Primeiro a chegar, primeiro a ser servido

First come, first served (FCFS)

- Processos são executados **na ordem que chegam** na fila de prontos
- O processo só libera o processador quando é **bloqueado** (ex. chamada de sistema) ou quando **termina**
- Ao retornar do estado bloqueado, o processo é inserido no **fim da fila**
- **Ex:** considere que processos chegam nesta ordem: A, B, C e D

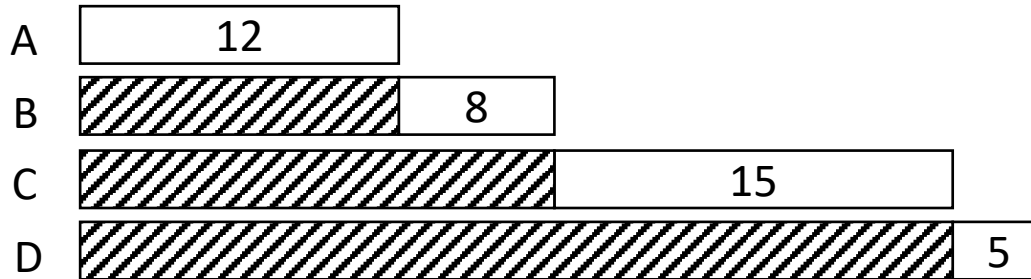


Pergunta: qual é o tempo médio de retorno?

Primeiro a chegar, primeiro a ser servido

First come, first served (FCFS)

■ Escalonamento



■ Cálculo do tempo médio de retorno

- TR_x = Tempo de retorno do processo x

$$TMR = \frac{TR_A + TR_B + TR_C + TR_D}{4}$$

$$TMR = \frac{12 + 20 + 35 + 40}{4} = 26,75$$

Primeiro a chegar, primeiro a ser servido

First come, first served (FCFS)

■ Implementação

- Uma única fila de processos prontos na ordem de chegada (**FIFO**)
- Processos são inseridos no fim da fila e removidos do início da fila

■ Vantagens

- Fácil de implementar e gerenciar

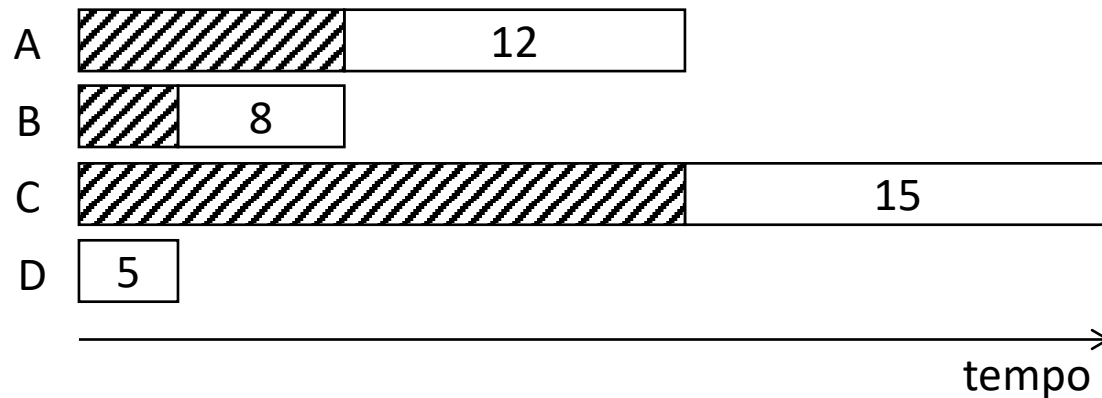
■ Desvantagem

- Não permite preempção
- Processos muito longos fazem com que os demais processos esperem muito tempo até ganhar o processador

Job mais curto primeiro

Shortest job first (SJF)

- Mantém os processos em uma **lista ordenada na ordem crescente de duração dos processos**
- O processo escolhido é sempre **o primeiro da lista**
- **Exemplo:**
 - **Processos:** A, B, C e D estão na fila de prontos
 - **Tempos:** A (12), B (8), C (15), D (5)

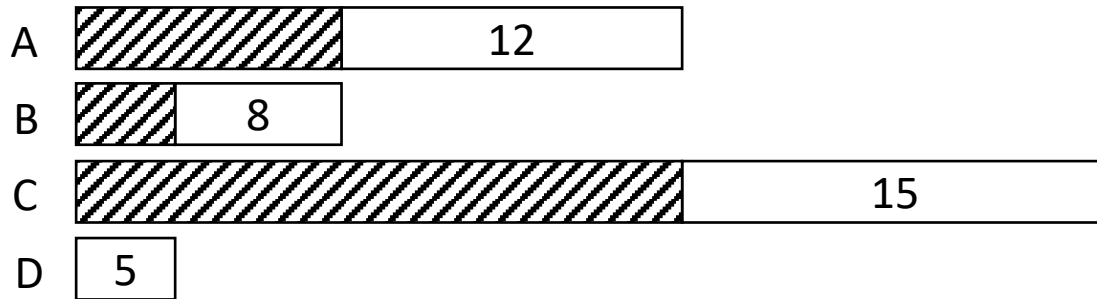


Pergunta: qual é o tempo médio de retorno?

Job mais curto primeiro

Shortest job first (SJF)

■ Escalonamento



■ Cálculo do tempo médio de retorno

- TR_x = Tempo de retorno do processo x

$$TMR = \frac{TR_D + TR_B + TR_A + TR_C}{4}$$

$$TMR = \frac{5 + 13 + 25 + 40}{4} = 20,75$$

Job mais curto primeiro

Shortest job first (SJF)

■ Implementação

- Uma única lista de processos prontos ordenada na ordem crescente do tempo de duração dos processos
- Processos removidos do início da lista

■ Vantagens

- Menor tempo médio de retorno
- Pode ser **preemptivo: Próximo de Menor Tempo Restante** (*Shortest Remaining Time Next*)

■ Desvantagens

- Precisa “prever o futuro” (técnicas de aproximação)
- **Postergação indefinida**

3

Escalonamento em sistemas interativos

Escalonamento por alternância circular

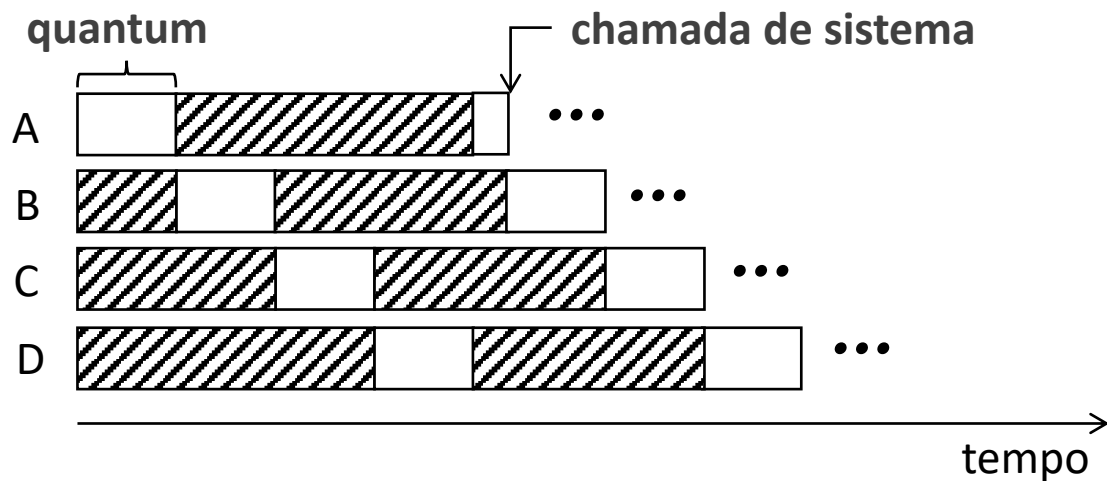
Round-robin

- Utiliza fila (FIFO) para controlar a ordem de execução
- A cada processo é atribuído um intervalo de tempo (***quantum***) no qual ele é permitido executar
- Processo executa até que:
 - Intervalo de tempo (*quantum*) esgote; **ou**
 - Seja bloqueado (ex: chamada de sistema)
- O processo é inserido no fim da fila quando retorna do estado bloqueado
- Necessita de um **relógio de tempo real**
 - Dispositivo de *hardware* usado para delimitar os intervalos de tempo por meio de **interrupções**

Escalonamento por alternância circular

Round-robin

- **Exemplo:** processos chegam nesta ordem: A, B, C e D



Escalonamento por alternância circular

Round-robin

- **Implementação**

- Uma lista circular
- Ponteiro para o processo atual (em execução)

- **Vantagens**

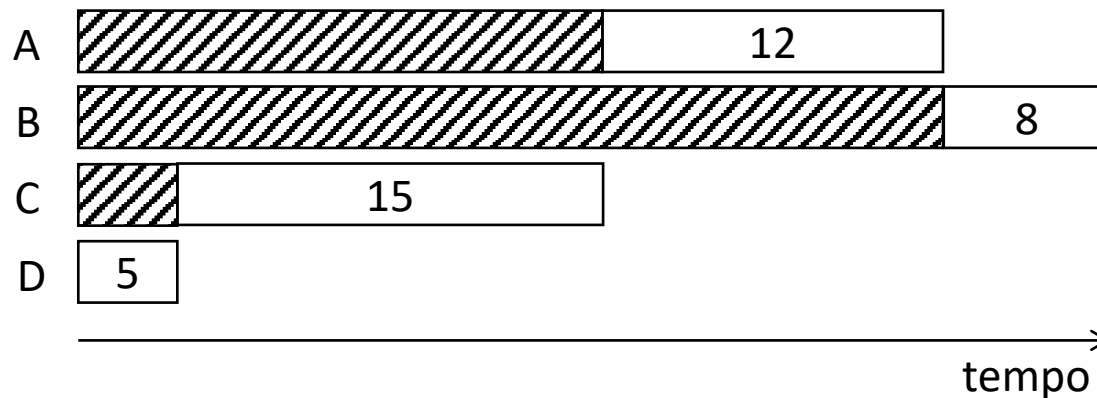
- Ilusão de paralelismo
- Minimiza o tempo de resposta

- **Problema:** como escolher o tamanho do *quantum*?

- **Quantum pequeno:** tempo desperdiçado com o chaveamento de contexto
- **Quantum grande:** perde-se a ilusão de paralelismo (resposta lenta às requisições interativas curtas)

Escalonamento por prioridades

- Atribui diferentes **prioridades** aos processos
- Prioridade define o processo a ser executado a seguir
- Usa **lista ordenada pela prioridade** (FIFO em caso de empate)
- **Exemplo**
 - **Tempos:** A (12), B (8), C (15) e D (5)
 - **Prioridades:** A (2), B (1), C (3) e D (4) – *maior executa primeiro*



Escalonamento por prioridades

- **Prioridades estáticas**

- Prioridade de um processo não muda durante o seu ciclo de vida

- **Prioridades dinâmicas**

- A prioridade de um processo é modificada dinamicamente

- **Exemplo de prioridade dinâmica**

- Processos que realizam muitas operações de E/S podem ter sua prioridade aumentada para iniciarem a requisição de E/S e então aguardarem no estado bloqueado enquanto outro processo é executado

Escalonamento por prioridades

■ Implementação

- Lista de processos ordenada pela prioridade
- Processos removidos do início da lista

■ Vantagens

- Permite atender processos importantes primeiro
- Pode usar **preempção**: um processo em execução na CPU pode ser preemptado quando um outro processo de mais alta prioridade entra para a fila de prontos

■ Desvantagem

- **Postergação indefinida**: processos de baixa prioridade podem nunca ser executados

Escalonamento por prioridades

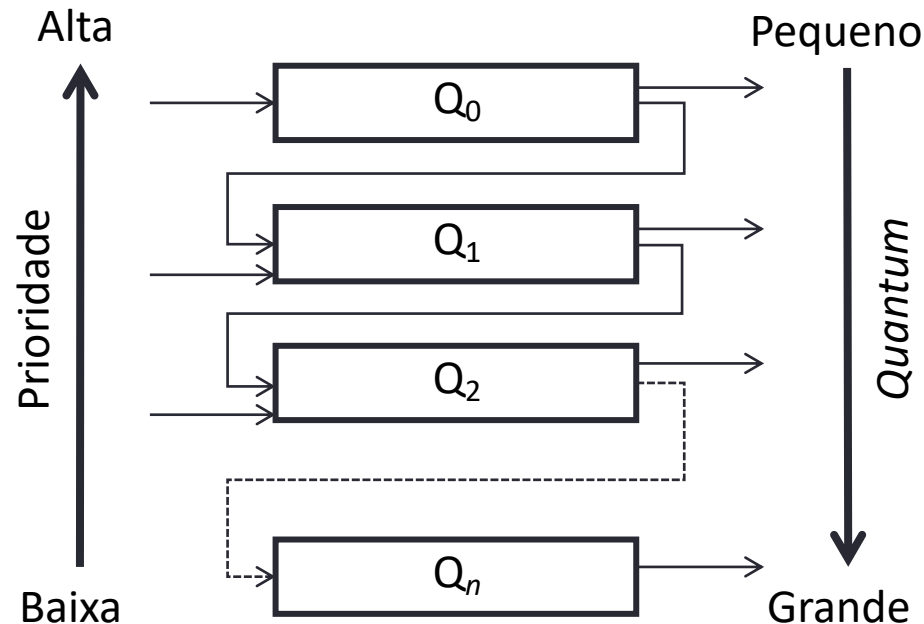
- **Solução para o problema da postergação indefinida**
 - Algoritmo de envelhecimento (*Aging*)
- **Funcionamento**
 - Processos que não ganham o processador tem a **prioridade aumentada gradativamente**
 - Em um dado momento, a prioridade do processo ficará **alta** e ele será **selecionado pelo escalonador**

Escalonamento com filas múltiplas

- Possui **diversas filas de processos** no estado pronto
- Cada processo é associado a uma **única fila**
- Cada fila pode utilizar um mecanismo próprio de escalonamento
- Cada fila possui uma **prioridade** associada
- **Duas alternativas**
 - **Sem realimentação:** uma vez associado a uma fila, o processo permanece sempre na mesma fila
 - **Com realimentação:** processos podem ser transferidos de fila de acordo com algum critério

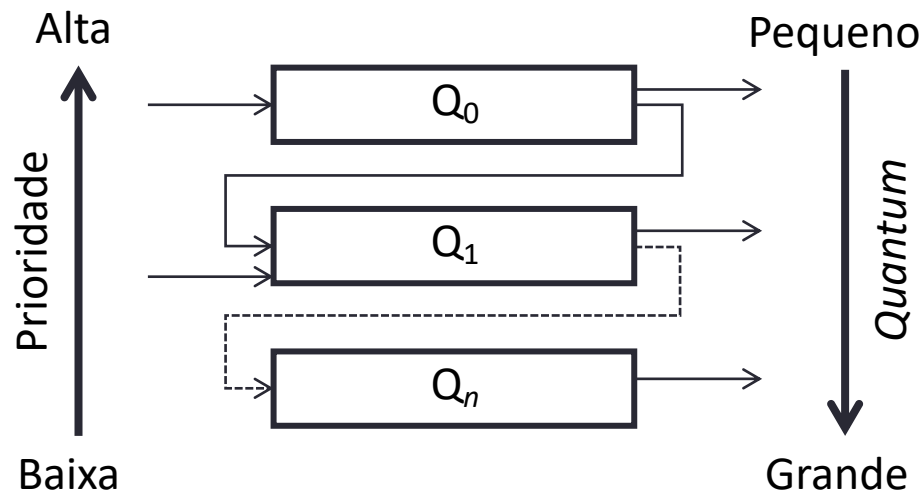
Escalonamento com filas múltiplas

- **Diversas filas:** $Q_0, Q_1, \dots, Q_n$
- **Prioridade:** diminui em direção à fila Q_n
 - Q_0 (alta), ..., Q_n (baixa)
- **Quantum:** aumenta em direção à fila Q_n
 - Q_0 (pequeno), ..., Q_n (grande)



Filas múltiplas com realimentação (exemplo)

- Processos em uma fila Q_i só podem ser escalonados se não existirem processos nas filas de maior prioridade ($Q_{i-1}, \dots, Q_0$)
 - Preempção quando novo processo é inserido em uma fila mais prioritária
- Ao esgotar seu *quantum* na fila Q_i , o processo é inserido na fila Q_{i+1}
- Quando bloqueado, o processo deixa a fila Q_i
 - Ao retornar, é inserido na fila Q_{i-1}



Filas múltiplas com realimentação (exemplo)

Vantagens de uma política como esta:

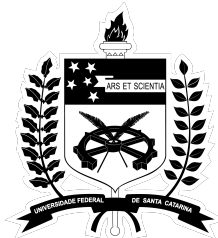
- Processos que utilizam muito a CPU tendem a descer em direção às filas de baixa prioridade, porém recebem *quanta* maiores (reduz custo de chaveamento)
- Processos interativos pequenos que realizam muitas operações de E/S são favorecidos, reduzindo-se o tempo médio de resposta do sistema
- **Problema: postergação indefinida**
 - Processos podem sempre ficar em filas de baixa prioridade devido a existência de processos em filas de alta prioridade
 - **Solução:** mecanismo de *aging* para mover gradativamente processos que estão em filas de prioridades baixas para filas de prioridade mais altas

! Obrigado pela atenção!



Dúvidas? Entre em contato:

- marcio.castro@ufsc.br
- www.marciocastro.com



**UNIVERSIDADE FEDERAL
DE SANTA CATARINA**



Distributed Systems Research Lab
www.lapesd.inf.ufsc.br